Oficina Mecânica — Fase 4

Microsserviços, Saga Pattern orquestrado e CI/CD
Tech Challenge SOAT · 11/2026

Participantes

Nome	RM	GitHub
[Nome do participante]	[RM]	[github]
[Nome do participante]	[RM]	[github]
[Nome do participante]	[RM]	[github]

Vídeo de demonstração

[link do vídeo no YouTube/Vimeo — até 15 minutos]

Roteiro: fluxo completo de uma OS pelos três microsserviços, execução da saga e das compensações, deploy automatizado de um microsserviço com validação de testes e rastreamento do fluxo distribuído (correlation id no Grafana/Loki e histórico da saga).

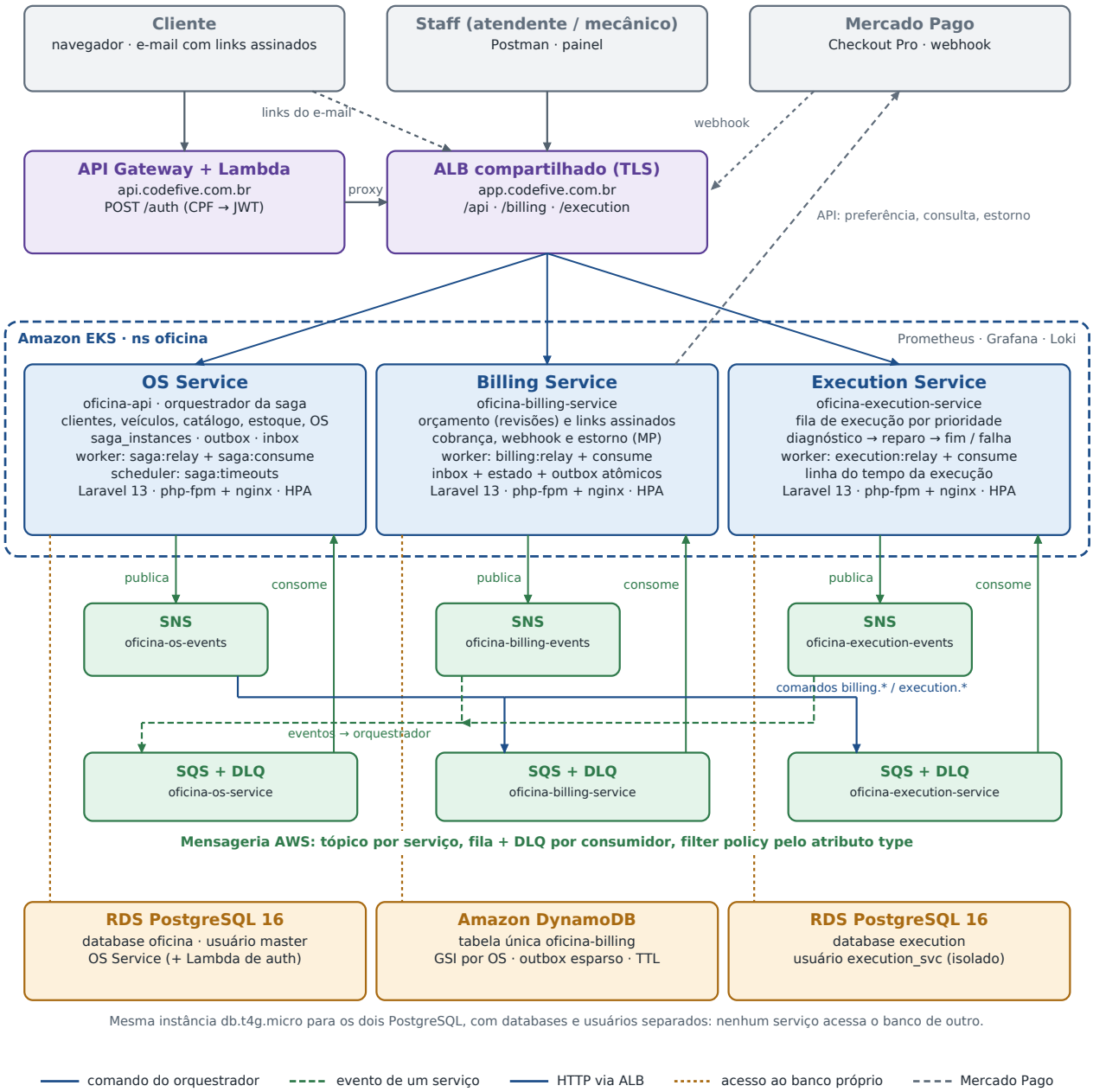
Repositórios

Repositório (github.com/MicaelHernandes/...)	Papel
oficina-api	OS Service — clientes, veículos, catálogo, estoque e OS; orquestrador da saga
oficina-billing-service	Billing Service — orçamento e pagamento (Mercado Pago)
oficina-execution-service	Execution Service — fila de execução, diagnóstico e reparo
oficina-k8s-infra	Infraestrutura base: VPC, EKS, ECR, ALB, DNS/TLS, observabilidade, OIDC
oficina-db-infra	RDS PostgreSQL e credenciais por serviço
oficina-auth-lambda	Autenticação do cliente por CPF (Lambda + API Gateway)

Cada microsserviço tem repositório, banco, infraestrutura (Terraform em *infra/*), Dockerfile, manifestos Kubernetes, pipelines de CI/CD, Swagger e documentação próprios. A collection Postman do fluxo completo está em *oficina-api/docs/postman*.

1. Arquitetura geral

Três microsserviços Laravel no mesmo cluster EKS, expostos pelo ALB compartilhado por prefixo de caminho. Cada um tem banco próprio e conversa com os outros só por mensagens (SNS → SQS); a coordenação das transações da OS é feita pela saga orquestrada no OS Service.



2. Divisão dos microsserviços

Serviço	Responsabilidade	Banco	Por que separado
OS Service oficina-api	Abertura, status e histórico da OS; clientes, veículos, catálogo e estoque; diagnóstico e montagem do orçamento; **orquestrador da saga**.	PostgreSQL (database *oficina*)	É o dono do ciclo de vida da OS e da máquina de estados — o lugar natural do orquestrador. Catálogo e estoque ficam junto porque a reserva de peças é um passo local da saga, consistente na mesma transação.
Billing Service oficina-billing-service	Orçamento oficial (revisões), envio ao cliente com links assinados, aprovação/recusa, cobrança no Mercado Pago, webhook e estorno.	DynamoDB (tabela *oficina-billing*)	Integração externa de pagamento com ciclo e falhas próprios (webhooks, reenvios, estornos). Isolada, uma instabilidade do gateway não derruba a abertura nem a execução de OS.
Execution Service oficina-execution-service	Fila de execução por prioridade; diagnóstico, reparo, anotações; comunicar fim ou falha ao OS Service.	PostgreSQL (database *execution*, usuário próprio)	Trabalho do chão de oficina, com escala e ritmo diferentes do atendimento e da cobrança. Pode evoluir (agenda de mecânicos, boxes) sem tocar nos outros serviços.

SQL e NoSQL: o OS Service e o Execution usam PostgreSQL porque têm dados relacionais e transações locais (estoque, fila com bloqueio de linha). O Billing usa DynamoDB: orçamento e cobrança são documentos (itens, cliente e payload do gateway) acessados por chave ou por OS, sem joins; on-demand, sem instância nem pod no nó único do EKS, e com transações para o padrão inbox/outbox.

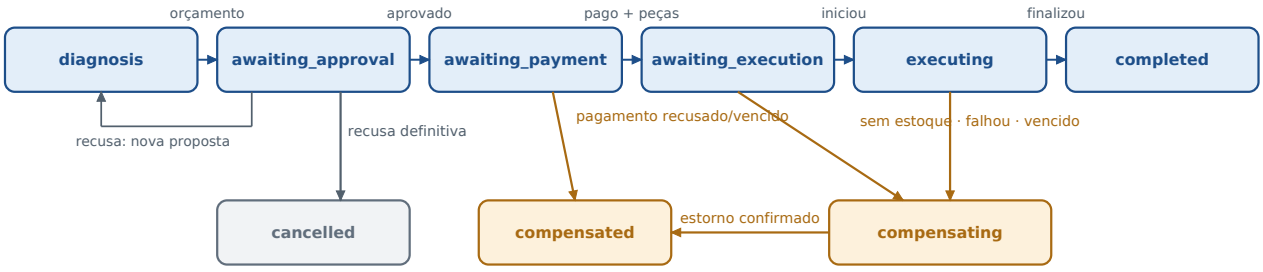
Isolamento dos dados: para não pagar uma segunda instância RDS no Free plan, o Execution tem database e usuário próprios na mesma instância. O Job de bootstrap revoga o CONNECT público dos dois databases: o usuário do Execution não conecta no database do OS Service, e nenhum serviço lê o banco de outro.

3. Tecnologias e justificativas

Tecnologia	Uso	Justificativa
PHP 8.4 / Laravel 13 (DDD)	Os três serviços	Mesma stack da Fase 3: reaproveita domínio, testes, Dockerfile, Kustomize e pipelines; o time domina.
Amazon EKS 1.34 + ALB	Execução e entrada HTTP	Cluster existente; um ALB compartilhado roteia /api, /billing e /execution sem novo certificado.
Amazon SNS + SQS	Mensageria da saga	Gerenciado e sem custo fixo; DLQ nativa, filter policy por tipo, IRSA. Broker no cluster disputaria o único nó.
RDS PostgreSQL 16	OS e Execution	Relacional com transações e SELECT FOR UPDATE; instância já provisionada (db.t4g.micro).
Amazon DynamoDB	Billing	NoSQL serverless on-demand; TransactWriteItems garante inbox + estado + outbox atômicos.
Mercado Pago (Checkout Pro)	Pagamento	Exigido pelo enunciado; webhook com x-signature e estorno idempotente.
Terraform + GitHub Actions (OIDC)	Infra e CI/CD	Cada serviço aplica a própria infra/ no deploy, sem chaves estáticas na AWS.
Pest · Behat · SonarCloud	Qualidade	Testes unitários e de feature com cobertura ≥ 80% no CI, BDD em Gherkin e análise estática no PR.
Prometheus · Grafana · Loki	Observabilidade	Stack da Fase 3; métricas de saga/outbox e logs JSON correlacionados.

4. Saga Pattern: estratégia orquestrada

A saga de cada OS é coordenada por um **orquestrador central no OS Service** (*OsSagaOrchestrator*). Ele mantém o estado em *saga_instances*, envia **comandos** ao Billing e ao Execution e reage aos **eventos** que eles publicam. O pagamento é cobrado na aprovação do orçamento, antes da execução; por isso a compensação de qualquer falha posterior é devolver as peças e estornar.



Por que orquestração e não coreografia:

- O fluxo da OS e todas as compensações ficam num só lugar, legíveis e testáveis (testes de feature e BDD). Na coreografia a lógica ficaria espalhada pelos três serviços e o rollback seria implícito.
- A OS já tinha uma máquina de estados central (*OsStatus*); a saga a estende em vez de duplicá-la.
- Rastreamento simples: *GET /api/order-services/{id}/saga* mostra o estado, o contexto e o histórico de cada passo.
- Custo aceito: o OS Service é ponto central do fluxo. Se cair, a saga pausa, mas as mensagens esperam na fila (4 dias) e o outbox retoma a publicação.

Passos e compensações

Passo	Comando / evento	Falha	Compensação
Abertura da OS	os.order.opened	—	—
Orçamento	billing.budget.generate → billing.budget.generated	Cliente recusa	Nova proposta (revisão) ou OS cancelada (cancelled)
Aprovação	billing.budget.approved → billing.payment.request	—	—
Pagamento	billing.payment.confirmed	Recusado ou vencido (48 h)	Cancela orçamento/cobrança e a OS (compensated)
Reserva de peças	local, tudo ou nada	Estoque insuficiente	Estorno (billing.payment.refund) e OS cancelada
Execução	execution.job.enqueue → started → finished	Falhou ou não iniciou (24 h)	Devolve peças, estorna, OS cancelada; fecha com billing.payment.refunded

Garantias de consistência

- **Transactional outbox:** o comando/evento é gravado na mesma transação da mudança de estado e publicado no SNS depois do commit (*relay*). Nenhuma mensagem sai sem commit, e nenhum commit fica sem a sua mensagem.
- **Inbox / idempotência:** o SQS entrega pelo menos uma vez; cada serviço registra o id da mensagem e ignora repetições. O Billing grava inbox, estado e outbox numa única *TransactWriteItems* do DynamoDB.
- **Lock da saga** (SELECT ... FOR UPDATE) por mensagem, e mensagens fora de ordem são ignoradas com log.
- **Timeouts** (*saga:timeouts*, a cada minuto) e **DLQ** após 5 tentativas para mensagens que falham sempre.

5. Comunicação entre microsserviços

- **Síncrona (REST/JSON):** só nas bordas — staff e cliente falam com cada serviço pelo ALB; o Billing chama a API do Mercado Pago e recebe o webhook. Nenhum serviço chama outro por HTTP no fluxo da saga.
- **Assíncrona (SNS → SQS):** cada serviço publica no próprio tópico e consome a própria fila. As filas do Billing e do Execution assinam o tópico do OS Service com filter policy (*billing.** / *execution.**); a fila do OS Service assina os tópicos dos dois.
- **Envelope comum:** id, type, saga_id, correlation_id, os_id, source, occurred_at e payload; tópicos criptografados (KMS) e acesso por IRSA, sem chaves estáticas.

6. Testes, qualidade e CI/CD

Serviço	Testes	Cobertura	Pipeline
OS Service	187 (Pest) + BDD: 5 cenários / 84 passos (Behat)	88%	Pint · PHPStan · Pest · Behat · Terraform · Docker → deploy EKS
Billing	51 (Pest, DynamoDB Local)	97%	Pint · PHPStan · Pest · Terraform · Docker → deploy EKS
Execution	23 (Pest, PostgreSQL)	95%	Pint · PHPStan · Pest · Terraform · Docker → deploy EKS

- **BDD:** *features/saga_ordem_de_servico.feature* (Gherkin em português) cobre o caminho feliz até a entrega, falta de estoque com estorno, falha na execução, pagamento recusado e

renegociação, contra o app real (HTTP + PostgreSQL).

- **Cobertura mínima de 80%** verificada no CI de cada serviço (*--coverage --min=80*), com o relatório como artefato; **SonarCloud** com Quality Gate no PR.
- **Deploy:** merge na *master* aplica a *infra/* do serviço, publica a imagem no ECR, cria Secrets/ConfigMaps a partir do Secrets Manager/SSM, roda os Jobs de banco/migrations e aguarda o rollout. Branches protegidas: *homolog* → *master* só por PR com checagens obrigatórias.

7. Observabilidade e rastreamento

- O **X-Request-Id** da abertura da OS vira o **correlation_id** da saga e segue em todas as mensagens e logs JSON dos três serviços: um filtro no Loki mostra o fluxo inteiro da OS.
- **Prometheus** (ServiceMonitor por serviço): latência por rota, sagas por estado, outbox pendente, execuções por status, tempo médio de execução e webhooks do Mercado Pago; dashboards no Grafana.
- Estado e histórico de cada saga na API do OS Service; linha do tempo de cada execução no Execution Service.